SQL PROJECT PRESENTATION

Database Relationship Analysis using Joins

SQL project presentation
Customer • Product • Order • Payment tables
Database: MySQL

45-SLIDE PRESENTATION

Introduction

- A database stores related information in separate tables.
- A JOIN combines rows from two or more tables.
- This project studies customer orders, products and payments.

Project Overview

- Create four related tables: Customer, Product, Orders and Payment.
- Connect tables using primary keys and foreign keys.
- Use INNER JOIN, LEFT JOIN and RIGHT JOIN.
- Prepare order details, purchase history and reports.

Objectives

- Understand table relationships.
- Design a simple relational database.
- Write SQL queries using different JOIN types.
- Retrieve complete order and payment information.
- Generate useful business reports.

What is a Relational Database?

- A relational database stores data in tables.
- Each table contains rows and columns.
- Tables are connected through key columns.
- Example: one customer can place many orders.

Why Do We Need JOINs?

- Related information is usually stored in separate tables.
- JOINs bring the required information into one result.
- They reduce repeated data and support reporting.
- Example: show a customer's name with their order date.

Project Scenario

- An online shopping store records customers, products, orders and payments.
- A customer places an order for a product.
- The order records quantity, date and status.
- A payment is linked to the order.

Database Tables

- Customer – stores customer information.
- Product – stores product information.
- Orders – stores each purchase.
- Payment – stores payment details.
- The tables are connected using IDs.

Relationship Diagram

- Customer (1) —— (Many) Orders
- Product (1) —— (Many) Orders
- Orders (1) —— (Many) Payment records
- Orders contains Customer_ID and Product_ID as foreign keys.

Primary Key and Foreign Key

- Primary Key (PK): uniquely identifies each row.
- Foreign Key (FK): refers to a key in another table.
- Customer_ID is the PK in Customer.
- Customer_ID in Orders is an FK.

Create the Database

```
CREATE DATABASE ShopDB;
USE ShopDB;
The first command creates the database.
USE selects the database for the following SQL commands.
```

Create Customer Table

```
CREATE TABLE Customer (
  Customer_ID INT PRIMARY KEY,
  Customer_Name VARCHAR(50),
  Email VARCHAR(100),
  City VARCHAR(40)
);
```

Create Product Table

```
CREATE TABLE Product (
  Product_ID INT PRIMARY KEY,
  Product_Name VARCHAR(60),
  Price DECIMAL(10,2),
  Category VARCHAR(40)
);
```

Create Orders Table

```
CREATE TABLE Orders (
  Order_ID INT PRIMARY KEY,
  Customer_ID INT,
  Product_ID INT,
  Order_Date DATE,
  Quantity INT,
  Status VARCHAR(30),
  FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
  FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
);
```

Create Payment Table

```
CREATE TABLE Payment (
  Payment_ID INT PRIMARY KEY,
  Order_ID INT,
  Payment_Date DATE,
  Amount DECIMAL(10,2),
  Payment_Mode VARCHAR(30),
  Payment_Status VARCHAR(30),
  FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
);
```

Insert Customer Records

```
INSERT INTO Customer VALUES
(1, 'Anu', 'anu@mail.com', 'Chennai'),
(2, 'Bala', 'bala@mail.com', 'Pondicherry'),
(3, 'Charan', 'charan@mail.com', 'Chennai'),
(4, 'Divya', 'divya@mail.com', 'Villupuram');
```

Insert Product Records

```
INSERT INTO Product VALUES
(101, 'Laptop', 45000, 'Electronics'),
(102, 'Mouse', 500, 'Accessories'),
(103, 'Keyboard', 1200, 'Accessories'),
(104, 'Phone', 18000, 'Electronics');
```

Insert Order Records

```
INSERT INTO Orders VALUES
(1001, 1, 101, '2026-10-01', 1, 'Delivered'),
(1002, 2, 102, '2026-10-03', 2, 'Shipped'),
(1003, 1, 103, '2026-10-05', 1, 'Delivered'),
(1004, 3, 104, '2026-10-08', 1, 'Processing');
```

Insert Payment Records

```
INSERT INTO Payment VALUES
(501, 1001, '2026-10-01', 45000, 'UPI', 'Paid'),
(502, 1002, '2026-10-03', 1000, 'Card', 'Paid'),
(503, 1003, '2026-10-05', 1200, 'Cash', 'Paid');
```

What is INNER JOIN?

- INNER JOIN returns only rows with matching values in both tables.
- Rows without a match are not included.
- It is useful when we need confirmed relationships.
- Example: orders that have a valid customer.

INNER JOIN: Customer and Orders

```
SELECT c.Customer_Name, o.Order_ID, o.Order_Date
FROM Customer c
INNER JOIN Orders o
ON c.Customer_ID = o.Customer_ID;
This displays each order with the customer's name.
```

INNER JOIN: Orders and Product

```
SELECT o.Order_ID, p.Product_Name,
       o.Quantity, p.Price
FROM Orders o
INNER JOIN Product p
ON o.Product_ID = p.Product_ID;
```

INNER JOIN: Order and Payment

```
SELECT o.Order_ID, p.Amount, p.Payment_Mode,
       p.Payment_Status
FROM Orders o
INNER JOIN Payment p
ON o.Order_ID = p.Order_ID;
```

What is LEFT JOIN?

- LEFT JOIN returns every row from the left table.
- Matching rows from the right table are added.
- If no match exists, right-table columns show NULL.
- It helps find customers who have not ordered.

LEFT JOIN: All Customers and Orders

```
SELECT c.Customer_Name, o.Order_ID, o.Status
FROM Customer c
LEFT JOIN Orders o
ON c.Customer_ID = o.Customer_ID;
Customers without orders are also displayed.
```

Find Customers Without Orders

```
SELECT c.Customer_ID, c.Customer_Name
FROM Customer c
LEFT JOIN Orders o
ON c.Customer_ID = o.Customer_ID
WHERE o.Order_ID IS NULL;
This identifies customers with no matching order.
```

What is RIGHT JOIN?

- RIGHT JOIN returns every row from the right table.
- Matching rows from the left table are included.
- If no match exists, left-table columns show NULL.
- It is useful when all right-table records must appear.

RIGHT JOIN: Orders and Payments

```
SELECT o.Order_ID, p.Payment_ID, p.Amount
FROM Payment p
RIGHT JOIN Orders o
ON p.Order_ID = o.Order_ID;
All orders appear, including orders without a payment.
```

Compare the Three JOINs

- INNER JOIN – only matching rows.
- LEFT JOIN – all left-table rows plus matches.
- RIGHT JOIN – all right-table rows plus matches.
- Choose a JOIN based on which records must be kept.

Complete Order Details

- Combine Customer, Orders, Product and Payment.
- Display customer name, product, quantity and order date.
- Also display amount, payment mode and payment status.
- Use LEFT JOIN for payment so unpaid orders remain visible.

Complete Order Details: SQL

```
SELECT c.Customer_Name, o.Order_ID, o.Order_Date,
       p.Product_Name, o.Quantity, p.Price,
       pay.Amount, pay.Payment_Mode, pay.Payment_Status
FROM Orders o
JOIN Customer c ON o.Customer_ID = c.Customer_ID
JOIN Product p ON o.Product_ID = p.Product_ID
LEFT JOIN Payment pay ON o.Order_ID = pay.Order_ID;
```

Customer Purchase History

- Purchase history shows all orders made by a customer.
- It helps understand customer activity.
- It can show date, product, quantity and order status.
- Filter by Customer_ID to view one customer.

Purchase History: SQL

```
SELECT c.Customer_Name, o.Order_ID, o.Order_Date,
       p.Product_Name, o.Quantity, o.Status
FROM Customer c
JOIN Orders o ON c.Customer_ID = o.Customer_ID
JOIN Product p ON o.Product_ID = p.Product_ID
WHERE c.Customer_ID = 1
ORDER BY o.Order_Date DESC;
```

Total Orders by Customer

```
SELECT c.Customer_Name,
       COUNT(o.Order_ID) AS Total_Orders
FROM Customer c
LEFT JOIN Orders o ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name;
COUNT counts the matching order records.
```

Total Spending by Customer

```
SELECT c.Customer_Name,
       SUM(pay.Amount) AS Total_Spending
FROM Customer c
JOIN Orders o ON c.Customer_ID = o.Customer_ID
JOIN Payment pay ON o.Order_ID = pay.Order_ID
WHERE pay.Payment_Status = 'Paid'
GROUP BY c.Customer_ID, c.Customer_Name;
```

Product Sales Report

```
SELECT p.Product_Name,
       SUM(o.Quantity) AS Units_Sold
FROM Product p
JOIN Orders o ON p.Product_ID = o.Product_ID
GROUP BY p.Product_ID, p.Product_Name
ORDER BY Units_Sold DESC;
```

Payment Summary Report

```
SELECT Payment_Mode, Payment_Status,
       COUNT(*) AS Payment_Count,
       SUM(Amount) AS Total_Amount
FROM Payment
GROUP BY Payment_Mode, Payment_Status;
```

Orders by Status

```
SELECT Status, COUNT(*) AS Number_of_Orders
FROM Orders
GROUP BY Status
ORDER BY Number_of_Orders DESC;
This report compares delivered, shipped and processing orders.
```

Monthly Sales Report

```
SELECT YEAR(o.Order_Date) AS Sales_Year,
       MONTH(o.Order_Date) AS Sales_Month,
       SUM(pay.Amount) AS Monthly_Sales
FROM Orders o
JOIN Payment pay ON o.Order_ID = pay.Order_ID
WHERE pay.Payment_Status = 'Paid'
GROUP BY YEAR(o.Order_Date), MONTH(o.Order_Date)
ORDER BY Sales_Year, Sales_Month;
```

Using GROUP BY and Aggregate Functions

- COUNT() – counts rows.
- SUM() – adds numeric values.
- AVG() – calculates an average.
- MIN() and MAX() – find smallest and largest values.
- GROUP BY creates a summary for each group.

NULL Values in JOIN Results

- NULL means a value is missing or not available.
- LEFT JOIN and RIGHT JOIN can produce NULL values.
- Use IS NULL to find missing matches.
- Use COALESCE(column, 0) when a numeric report needs zero.

Common JOIN Errors

- Joining on the wrong columns.
- Forgetting the ON condition.
- Using ambiguous column names.
- Counting duplicate rows after joining one-to-many tables.
- Use table aliases such as c, o, p and pay.

Best Practices

- Use clear primary and foreign keys.
- Give tables and columns meaningful names.
- Use aliases to keep queries readable.
- Select only the columns required.
- Test JOIN queries with small sample data.

Conclusion

- SQL JOINs connect related information across tables.
- INNER, LEFT and RIGHT JOINs serve different purposes.
- Aggregate functions create useful summaries.
- This project demonstrates practical relational database analysis.

Thank You

Database Relationship Analysis using Joins
Questions?

SQL PROJECT PRESENTATION